

Week 11 Lab: IMU Dashboard with TFT and Character LCD

EE0827 Embedded System Applications

Overview

Build a small embedded dashboard using three devices:

- **MPU6050 IMU** over I2C
- **HD44780 16x2 character LCD** through an I2C backpack
- **ST7789 TFT display** over SPI

The TFT will show a simple moving square whose position follows the board tilt. The character LCD will print short roll/pitch messages. The code is intentionally simple: no RTOS, no TFT DMA, no interrupt-driven IMU sampling, and no complex architecture.

Time allocation: 2 hours

- Wiring and safety gate: 20 min
- CubeMX setup: 25 min
- Add starter code and build: 25 min
- Demo: IMU angle controls TFT shape and LCD text: 30 min
- Measurements and failure injection: 20 min

Learning objectives

1. Configure one I2C bus with two devices: MPU6050 and LCD backpack.
2. Configure SPI1 to drive an ST7789 TFT display.
3. Use simple driver APIs from `main.c` through `app_main()`.
4. Convert accelerometer data into simple roll/pitch angles.
5. Display the same physical measurement in two ways: graphical TFT shape and character LCD text.
6. Capture basic evidence: I2C voltage/timing and TFT update timing.

Safety gate

⚠ Stop before applying power

- ☐ All modules share **GND** with the Nucleo.
- ☐ MPU6050 is powered from **3.3 V**.
- ☐ TFT is powered from **3.3 V**.
- ☐ LCD backpack power is verified. If using 5 V, SDA/SCL voltage safety is verified or a level shifter is used.
- ☐ SDA and SCL are not swapped.
- ☐ TFT SCK and MOSI are not swapped.
- ☐ No module VCC is connected to an unknown pin.

⚠ Common ways to destroy the board

- Powering a 3.3 V-only TFT from 5 V.
- Pulling I2C SDA/SCL up to 5 V when the STM32 pins are not protected by a level shifter.
- Wiring module VCC to an MCU GPIO pin by mistake.

This procedure prevents these failures by using 3.3 V power by default and requiring a powered bus-voltage check before firmware testing.

Required parts/tools

Item	Qty	Notes
STM32 Nucleo-F401RE	1	Main board
MPU6050 / GY-521 module	1	I2C IMU
16x2 HD44780 LCD + I2C backpack	1	Usually PCF8574, address 0x27 or 0x3F
ST7789 240x240 TFT	1	SPI display
Breadboard + jumper wires	–	Keep wiring short
Oscilloscope or logic analyzer	1	For I2C and trace timing
Serial terminal	1	115200 baud UART2 log

Pre-lab

Answer before wiring:

1. If the MPU6050 and LCD backpack are on the same I2C bus, what makes their transactions separate?

2. The LCD backpack address is usually 0x27, but some modules use 0x3F. What should you do before changing driver code?

3. A 240 x 240 RGB565 TFT frame is how many bytes?

Procedure

Step 1.1: I2C bus wiring

Use I2C1 for both MPU6050 and LCD backpack.

Nucleo pin	Connect to	Notes
PB8 / D15	MPU6050 SCL, LCD SCL	I2C1_SCL
PB9 / D14	MPU6050 SDA, LCD SDA	I2C1_SDA
3V3	MPU6050 VCC, TFT VCC	3.3 V default
3V3 or verified safe supply	LCD backpack VCC	Prefer 3.3 V if contrast/backlight are acceptable
GND	all module GND pins	common ground required
MPU6050 AD0	GND	sets address to 0x68

Powered check before flashing code:

- SDA to GND: approximately 3.3 V when idle
- SCL to GND: approximately 3.3 V when idle

Step 1.2: TFT wiring

Nucleo pin	TFT pin	Notes
PA5 / D13	SCL / SCK	SPI1_SCK
PA7 / D11	SDA / MOSI	SPI1_MOSI
PA8	RES / RST	GPIO output, label TFT_RST
PA9	DC	GPIO output, label TFT_DC
3V3	VCC	3.3 V only unless module says otherwise
GND	GND	common ground

Optional measurement pin:

Nucleo pin	Label	Purpose
PA10	TRACE	Goes high during TFT redraw

Step 2.1: Create project

1. New STM32CubeIDE project.
2. Board selector: **NUCLEO-F401RE**.
3. Project name: **W11_IMU_Display_Demo**.
4. Keep default USART2 for serial terminal.

Step 2.2: Configure I2C1

- Connectivity -> **I2C1**
- Mode: I2C
- Pins: PB8 = SCL, PB9 = SDA
- Speed: **100 kHz** first
- Addressing: 7-bit

Step 2.3: Configure SPI1

- Connectivity -> **SPI1**
- Mode: Transmit Only Master, or Full-Duplex Master if Transmit Only is unavailable
- Data size: 8 bits
- First bit: MSB first
- CPOL: Low
- CPHA: 1 Edge
- NSS: Disabled
- Prescaler: 32 or 64 for first bring-up

Step 2.4: Configure GPIO

Pin	Mode	Label
PA8	GPIO output	TFT_RST
PA9	GPIO output	TFT_DC
PA10	GPIO output	TRACE

Step 2.5: USART2

- Baud: 115200
- 8 data bits, no parity, 1 stop bit

Generate code.

Part 3

Add simple drivers and app code

25 minutes

Step 3.1: Copy starter code

Starter code location:

```
weeks/W11_display-interfacing/lab/starter_code/w11_imu_display_demo/
```

Copy these folders into your CubeIDE project root:

```
app/  
drivers/  
bsp/
```

Add these include paths in CubeIDE if needed:

```
app  
drivers  
bsp
```

The MPU6050 driver uses `atan2f()` and `sqrtf()` for the simple tilt estimate. If CubeIDE reports linker errors for these functions, add the math library:

```
Project Properties -> C/C++ Build -> Settings -> MCU GCC Linker -> Libraries -> add m
```

Step 3.2: Modify main.c

Inside `main.c`, add:

```
/* USER CODE BEGIN Includes */  
#include "app.h"  
/* USER CODE END Includes */
```

After CubeMX initializes GPIO, I2C, SPI, and UART, call:

```
/* USER CODE BEGIN 2 */  
app_main();  
/* USER CODE END 2 */
```

`app_main()` never returns, so the CubeMX `while (1)` can remain empty.

Step 3.3: What the code does

The code path is deliberately short:

```
main.c
-> app_main()
  -> initialize LCD
  -> initialize MPU6050
  -> initialize TFT
  -> forever:
    read IMU roll/pitch
    draw moving square on TFT
    print roll/pitch on character LCD
```

Important files:

File	Purpose
app/app.c	Main demo loop
drivers/mpu6050_simple.c	Wake IMU and read accel-only roll/pitch
drivers/lcd_i2c_simple.c	HD44780 I2C backpack text output
drivers/st7789_simple.c	ST7789 init and simple rectangle drawing
bsp/bsp.c	Trace pin and UART helper

Simplicity rule

This lab is not about building a perfect display framework. It is about seeing how sensor data moves through simple drivers into visible outputs. Keep the code readable and make the demo work first.

Step 4.1: Expected serial output

Open a serial terminal at 115200 baud. Reset the board.

Expected startup line:

```
W11 IMU + displays demo
I2C devices found:
  0x27
  0x68
```

If the LCD is not found, the UART prints:

```
LCD init failed. Check address 0x27/0x3F.
```

Step 4.2: Expected TFT behavior

The TFT shows:

- black background
- blue center cross
- yellow square

Tilt the board:

- pitch changes horizontal square position
- roll changes vertical square position

Step 4.3: Expected character LCD behavior

The LCD shows roll and pitch values, updated slowly enough to read:

```
Roll: +12 deg
Pitch: -4 deg
```

If the text is missing but the backlight is on, adjust the backpack contrast potentiometer.

Measurements

Measurement 1: I2C bus idle voltage and address scan

Instrument: DMM and serial terminal.

Record:

Signal / result	Measured value
SDA idle voltage	
SCL idle voltage	
MPU6050 address found	
LCD backpack address used	

Pass criteria:

- SDA and SCL idle near 3.3 V.
- MPU6050 found at 0x68.
- LCD backpack responds at 0x27 or 0x3F.

Measurement 2: TFT redraw time

Instrument: oscilloscope on TRACE pin PA10.

The starter code sets TRACE high during TFT redraw and low after redraw.

Record:

Measurement	Value
TRACE high time for one redraw	
TFT update period	

Pass criteria:

- Shape moves visibly when the board is tilted.
- TRACE high pulse is repeatable.

Failure injection test

Test: wrong LCD backpack address

1. Change in `app/app.c`:

```
#define LCD_ADDR LCD_I2C_ADDR_27
```

to:

```
#define LCD_ADDR LCD_I2C_ADDR_3F
```

or reverse it, depending on your actual module.

2. Rebuild and flash.
3. Observe LCD behavior and UART message.
4. Restore the correct address.

Expected behavior:

- LCD does not initialize with the wrong address.
- UART reports that LCD initialization failed.
- Restoring the correct address recovers the display.

Deliverables checklist

- ☐ Photo of final wiring.
- ☐ Serial terminal screenshot or copied output.
- ☐ TFT shows a shape that moves with board tilt.
- ☐ Character LCD prints roll and pitch messages.
- ☐ I2C voltage/address measurement table completed.
- ☐ TRACE timing measurement completed.
- ☐ Failure injection result documented.

Grading checklist

Category	Evidence
Demo correctness	IMU controls TFT square; LCD prints messages
Measurements/evidence quality	I2C voltage/address table and TRACE timing
Design rationale	Can explain why one I2C bus can host IMU + LCD
Code understanding	Can explain <code>app_main()</code> and the three simple drivers
Failure analysis	Wrong LCD address test documented and recovered